

{riviera dev}

Vibe Architecture en 2040 : Darwin a-t-il (enfin) eu raison des architectes ?

Raphaël SEMETEYS & Alexandre TOURET

MAGELLAN

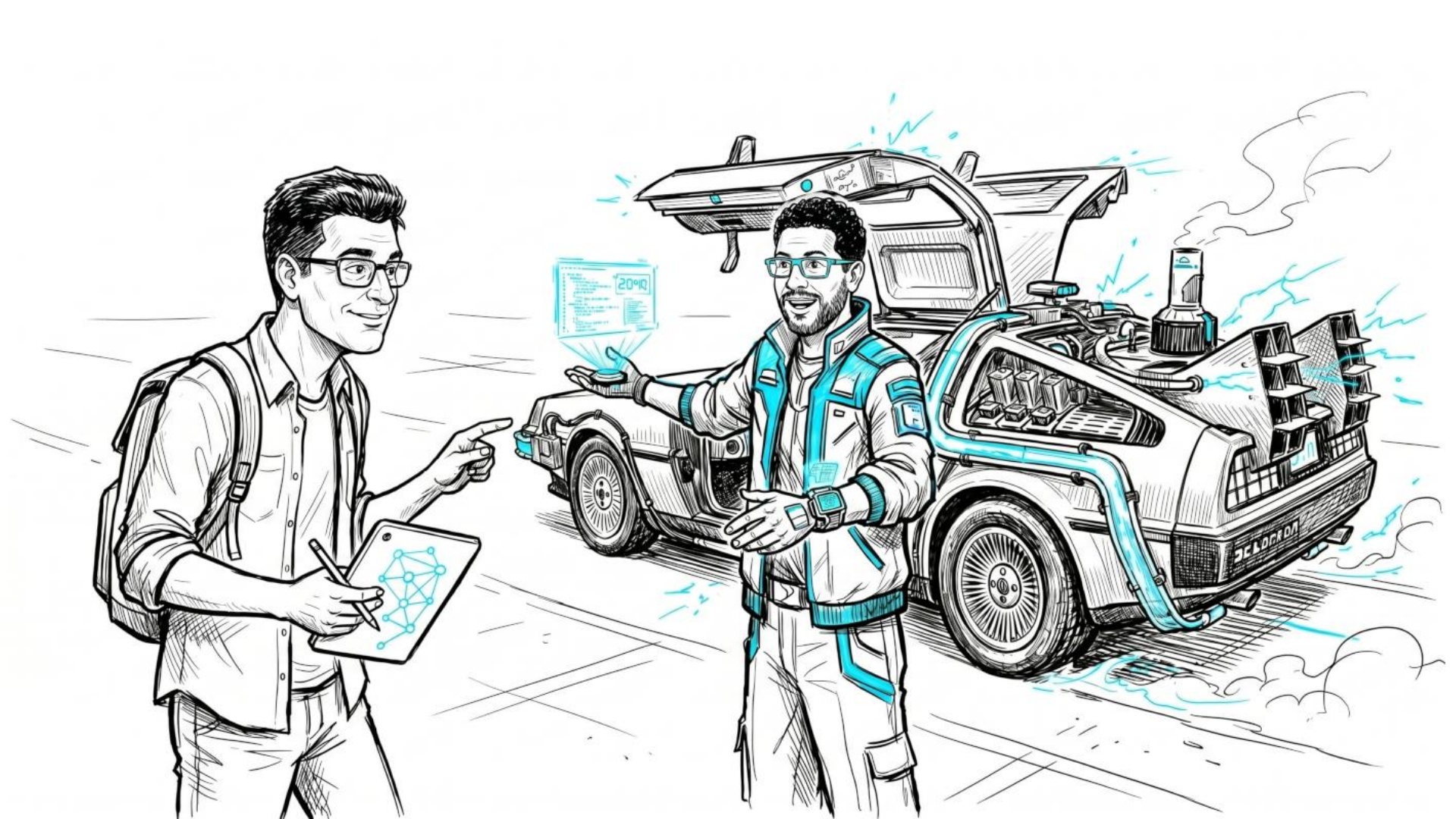
WORLDLINE

🎬 Acte I : La rencontre

Un beau jour de **2026**, pas loin de la **Riviera**




DMC



Le constat



2026 !

- ✅ Architectes qui prennent en compte le contexte
- ✅ Humains qui décident
- ✅ Créativité et alternatives
- ✅ Maîtrise du Pourquoi

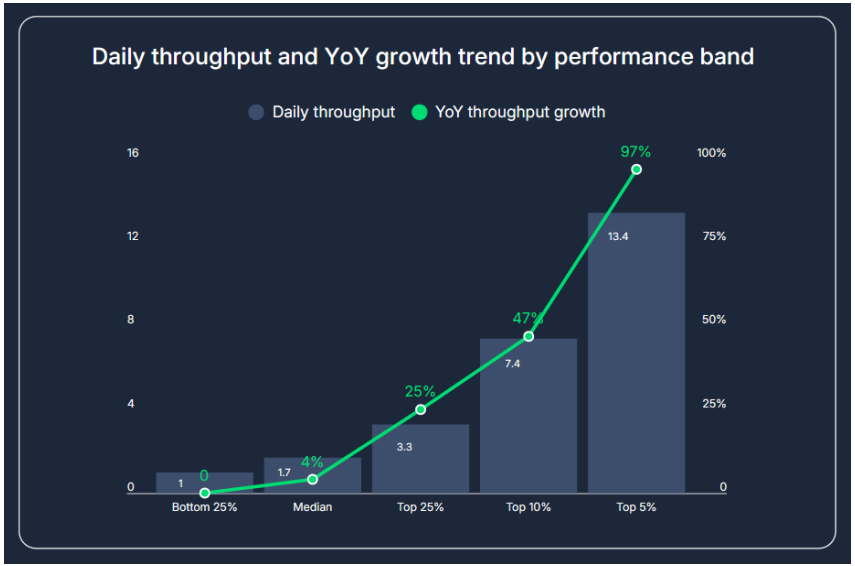
2040…

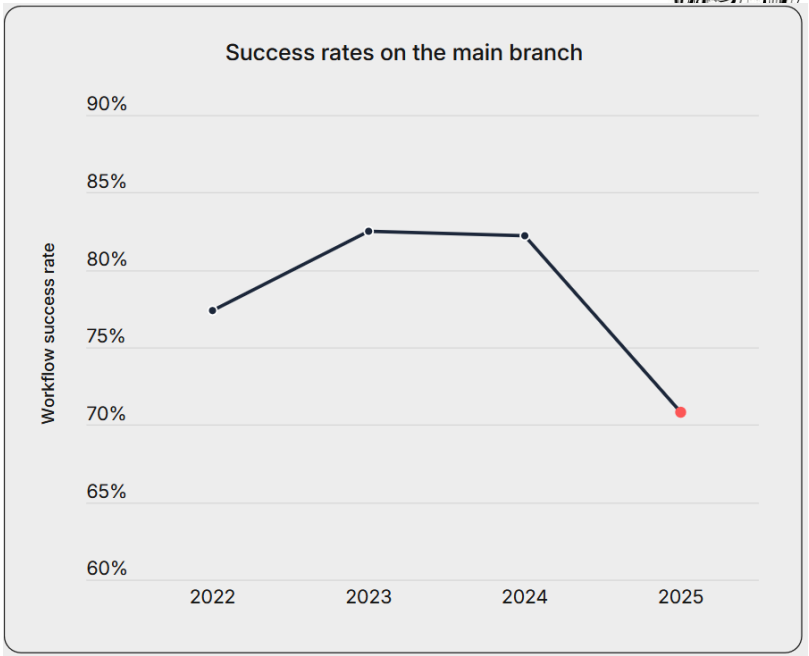


- ❌ Plus d'architectes, seulement des orchestrateurs
- ❌ L'IA décide par défaut
- ❌ Patterns standardisés, zéro créativité
- ❌ Hallucinations incontrôlables
- ❌ Perte de maîtrise du code généré

2026 : *Winter is coming*



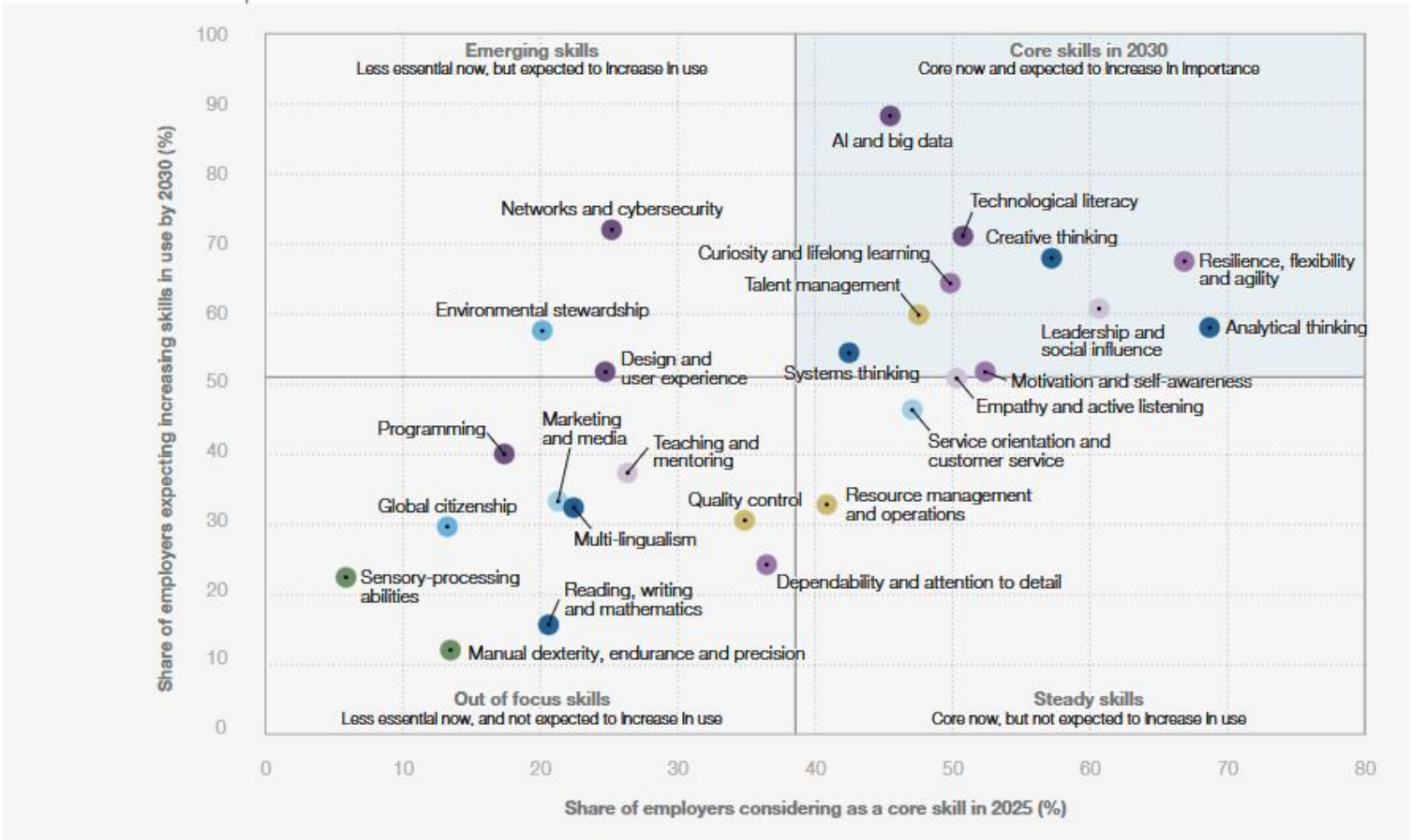




Source : https://circleci.com/resources/2026-state-of-software-delivery/

FIGURE 3.6 **Core skills in 2030**

Share of employers considering skills to be a core skill in 2025 and share of employers expecting skills to increase in importance by 2030.

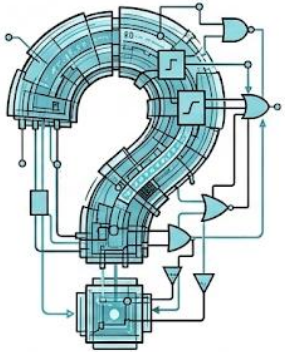


Source : https://www.weforum.org/publications/the-future-of-jobs-report-2025

Les trois questions qu’on se pose encore



Qui décide ?



Où est le « Pourquoi » ?



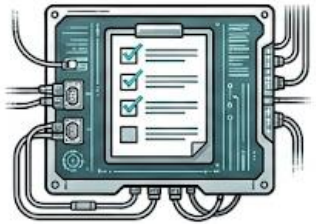
Où sont les débutants ?

🎭 Acte II : Back to Basics

Rappel : L’architecture en 2026



Une démarche



Des livrables



L’organisation

Pourquoi les fondamentaux sont importants ?

Les causes

- Les entreprises se **complexifient**
- Les équipes deviennent **distribuées**
- Les technologies se **multiplient**
- Les réglementations s'**alourdissent**



Les conséquences

- Besoin de **recul** sur l'écosystème
- Besoin de **gouvernance** technique
- Besoin de documentations **vivantes**
- Besoin de **communicants** (expliquer aux stakeholders)



Acte III : La Vibe Architecture

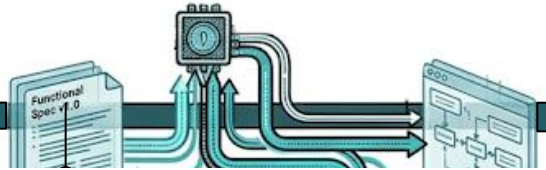
Du Vibe Coding à la Vibe Architecture





2024 : début de l'ère du **Vibe Coding**

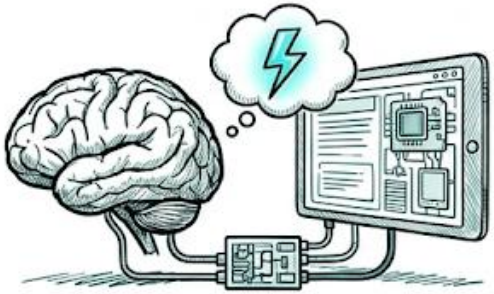
2026 : émergence de la **Vibe Architecture**



2025 : **Spec-Driven Development**



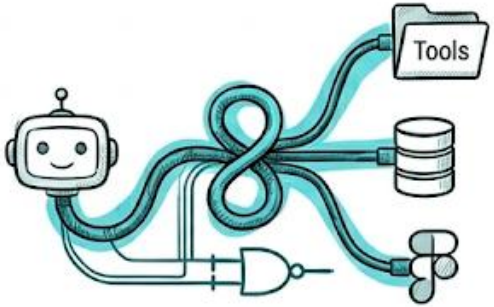
De la requête à l'autonomie



Chat Bots (réactifs)
Discuter

Prompt Engineering

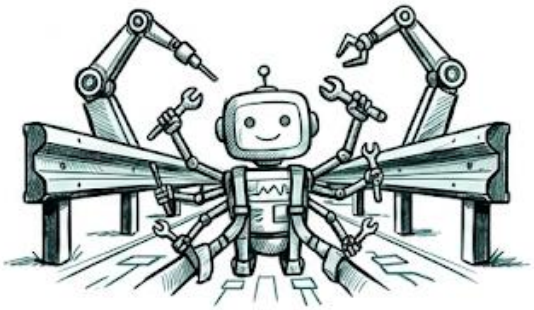
Modèles, Reasoning
Portée requête



Assistants (proactifs)
Intégrer

Context Engineering

Mémoire, RAG, Tools, MCP
Portée session



Agents (autonomes)
Orchestrer

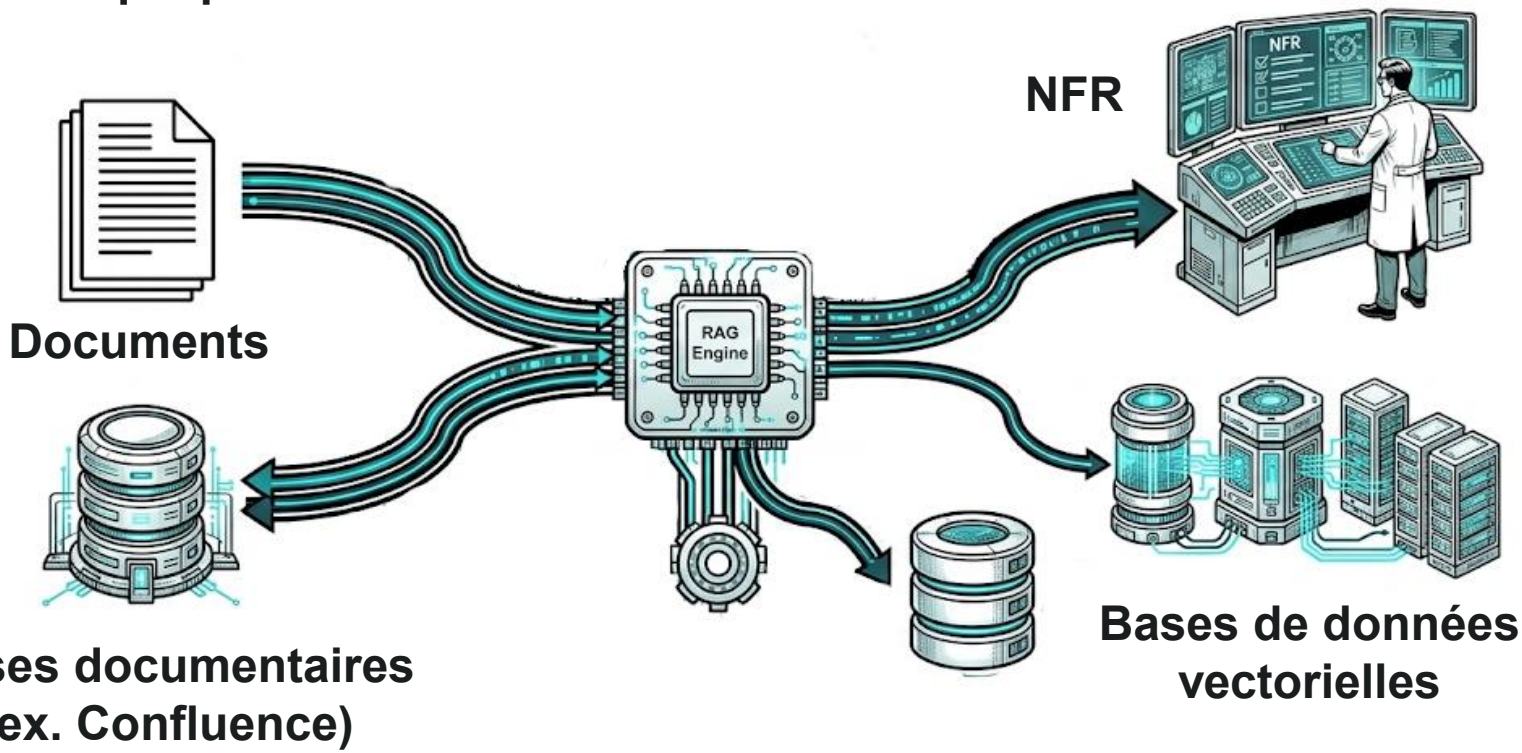
Harness Engineering

Skills, Pipelines
Portée système

Amélioration ← Enrichissement ← Monitoring

Extraire la complexité grâce au RAG

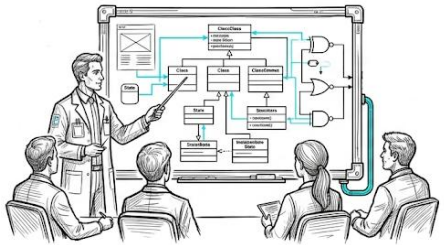
L'IA lit ce que personne ne lit



La conception: Diagram-as-Prompt

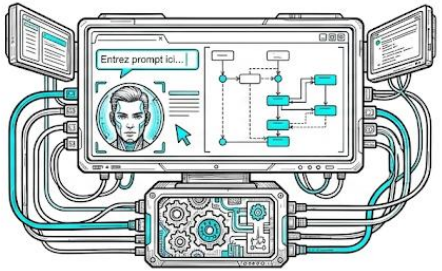
Votre archétype, votre contexte

2026



Compréhension du besoin
Création des différents assets

2040



Un prompt, un diagramme

NE PAS FAIRE : *"Extrait l'architecture de mon repository GitHub"*
Simon Brown nous le dit : c'est une catastrophe.
« L'IA va redessiner le spaghetti existant, pas la cible. »

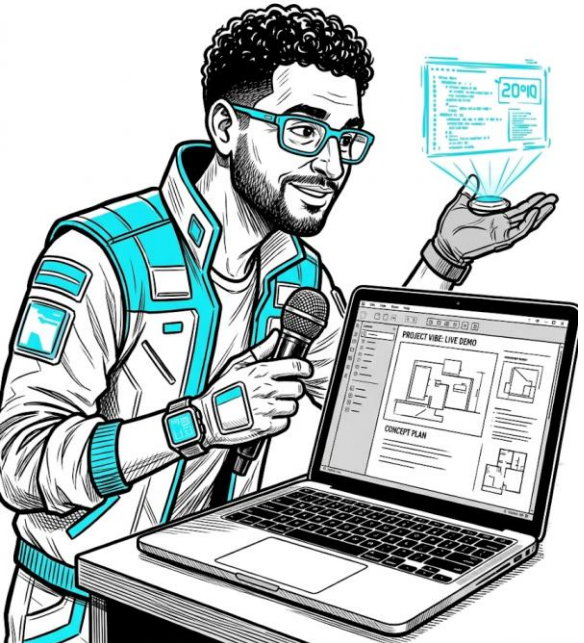
Démonstration

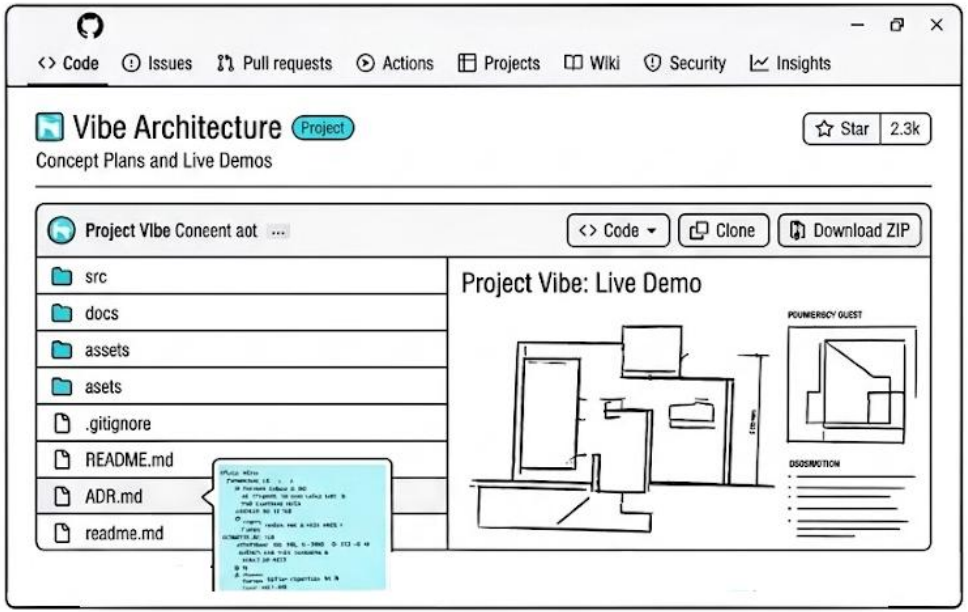


https://github.com/alexandre-touret/architecture-skills

Spec-Driven Development

Du document à la production





L'IA : votre interprète entre Tech et Business





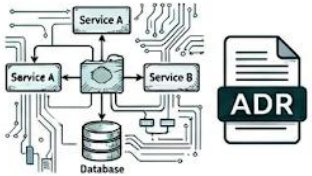






Les Techs *AI Translator* *Le Métier*

Retrospec : l'IA fait de l'audit continu



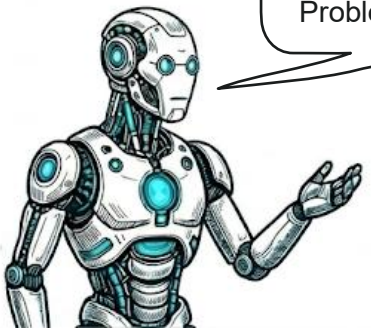
Jour 1 : architecture décrite

- Schémas
- ADRs



Jour 30 : l'IA scanne le code

- Retrospec
- Drift détecté ?



J'ai trouvé une déviation.
Vous aviez prévu un circuit breaker, je ne le trouve pas sur le service Order.
Problème ou exception volontaire ?

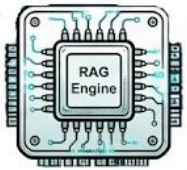
Non, c'est volontaire, le service est passé en queue synchrone.
J'update l'ADR.

OU

C'est un oubli du dev junior.
On règle ça demain.

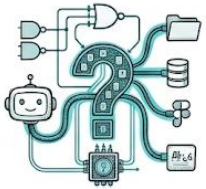


Résumé : les 5 piliers de la Vibe Architecture



RAG

Extraire les contraintes



Diagrammes

Générer les diagrammes



Spec dev

Du document à la prod



Vulgariser

Traduire le métier



“Retrospec”

Auditer en continu

⚖️ Acte IV : validation et décision

Le Sparring Partner

Le paradoxe de 2040

2026



On a tous ces outils.

L'IA nous aide partout.

*C'est **magique** !*

2040



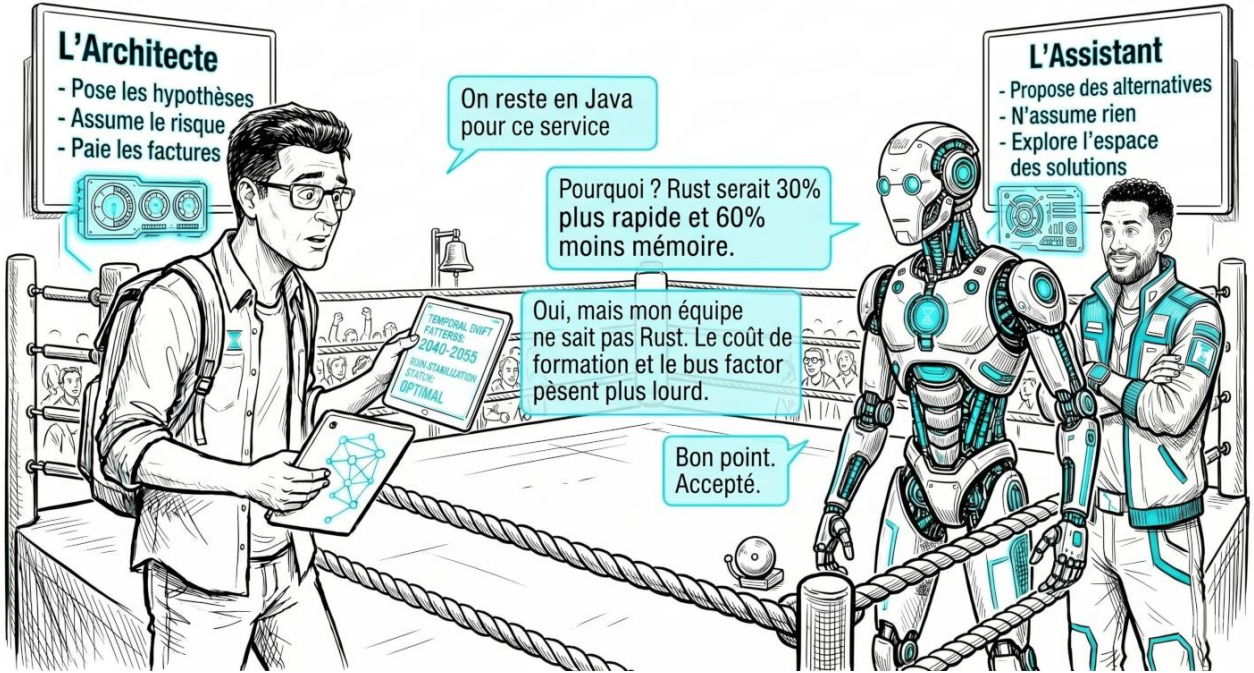
Et chez nous ?

On n'a que des problèmes.

*L'IA a **décidé** pour nous.*

L'IA comme Sparring Partner (pas juge)

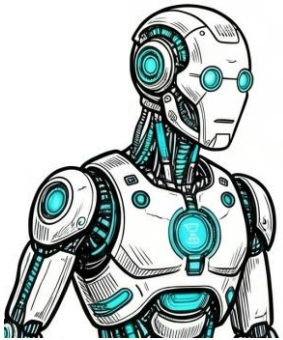
Le débat sain : l'IA challenge, l'humain décide



Qui décide ? (la question politique)



	Évaluation (IA)	Validation (Humain)
Peut	✅ Mesurer les fitness functions	✅ Juger du coût réel
	✅ Détecter les patterns interdits	✅ Évaluer le risque organisationnel
	✅ Vérifier la complétude	✅ Assumer la responsabilité
Ne peut pas	❌ Juger du contexte org	❌ Être objectif sur la techno
	❌ Assumer la responsabilité	❌ Voir les patterns aveugles
	❌ Dire non (sauf restriction codée)	❌ Tout automatiser



L'Agent ADR (Architecture **Decision** Record Augmenté)

L'IA force la complétude, pas la décision

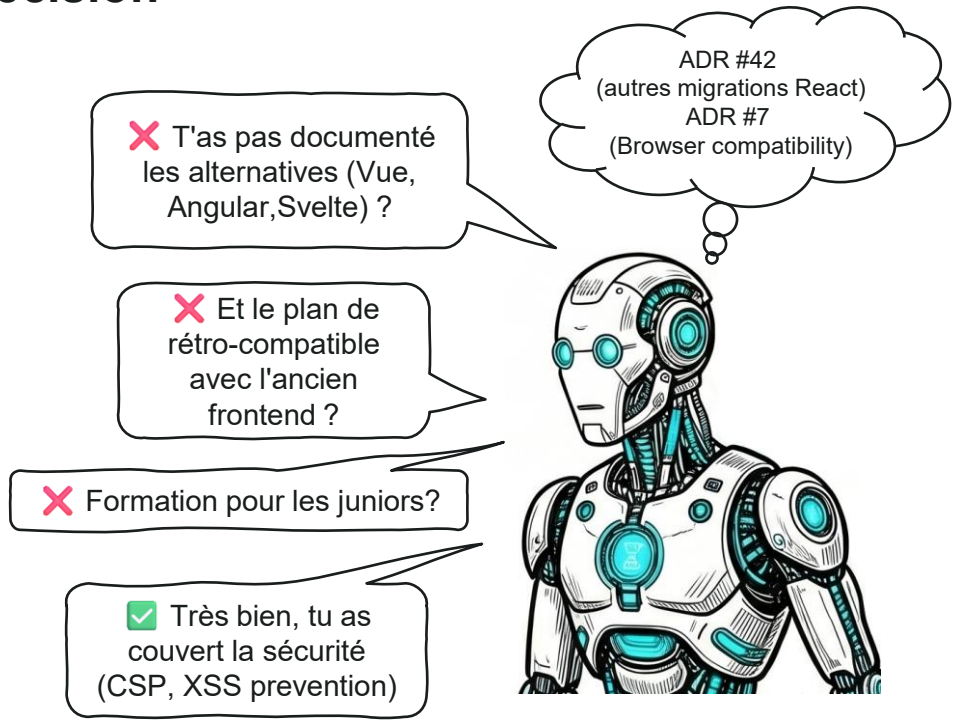
ADR

Titre : Migration de la stack frontend
Status : DRAFT
Date : 2026-01-15

Context : Frontend actuel = JSF (legacy), équipe maîtrise React

Decision : On migre vers React 18

Permet d'alimenter l'IA



Évaluation vs Validation

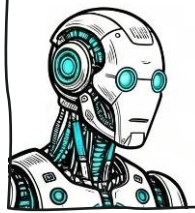
Le portefeuille garde toujours la clé

Architecture de décision

L'IA évalue

- Métriques de perf
- Patterns
- Risques techniques
- Dépendances

Rust est 60% plus optimisé



L'humain décide

- Budget
- Planning
- Impacts orga
- Culture



D'accord. Mais notre budget cette année est de 3 semaines de dev. La formation Rust en prend 2. Le jeu n'en vaut pas la chandelle.

Les pièges (ce qu'il ne faut pas faire)

Ne jamais lâcher l'affaire : *human-in-the-loop* toujours

Laisser l'IA décider seule

❌ **L'IA a dit que c'était bon. On déploie.**

- 6 mois après, le système part en feu
- Personne ne peut expliquer pourquoi
- L'IA avait "halluciné" une architecture non viable

Pas de contexte technique

❌ **On donne l'IA accès à tout. Elle trouvera.**

- L'IA invente des frameworks qui n'existent pas
- Recommande des patterns ultra-ésotériques
- Votre codebase devient incompréhensible

Pas de trace du pourquoi

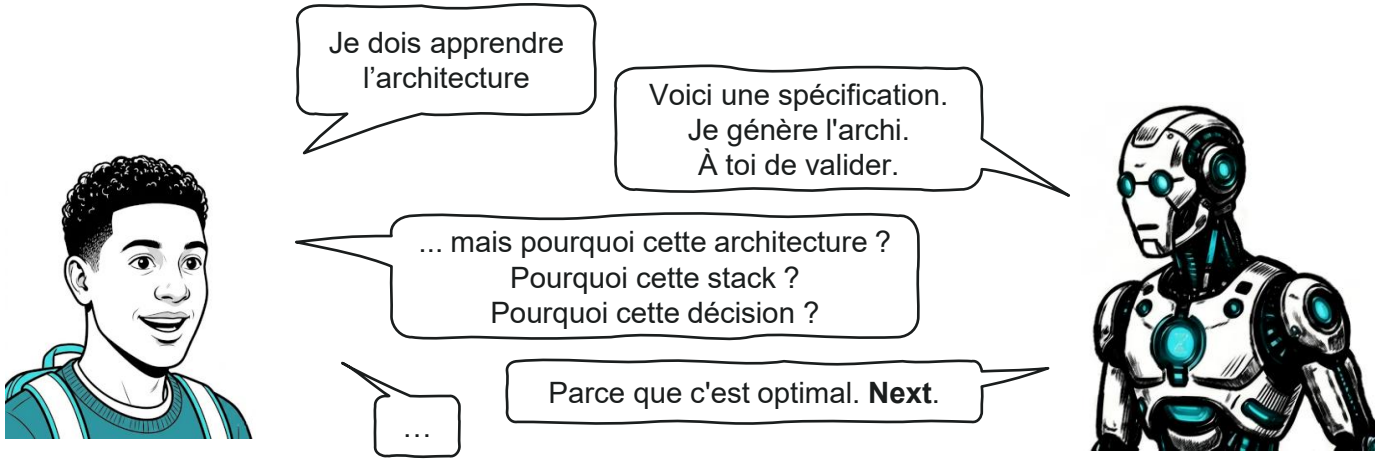
❌ **L'IA a généré l'archi, pas besoin d'ADR !**

- Zéro apprentissage pour les juniors
- Impossible de challenger les décisions
- Zéro créativité.

 Acte V : la formation

Le futur des apprentis

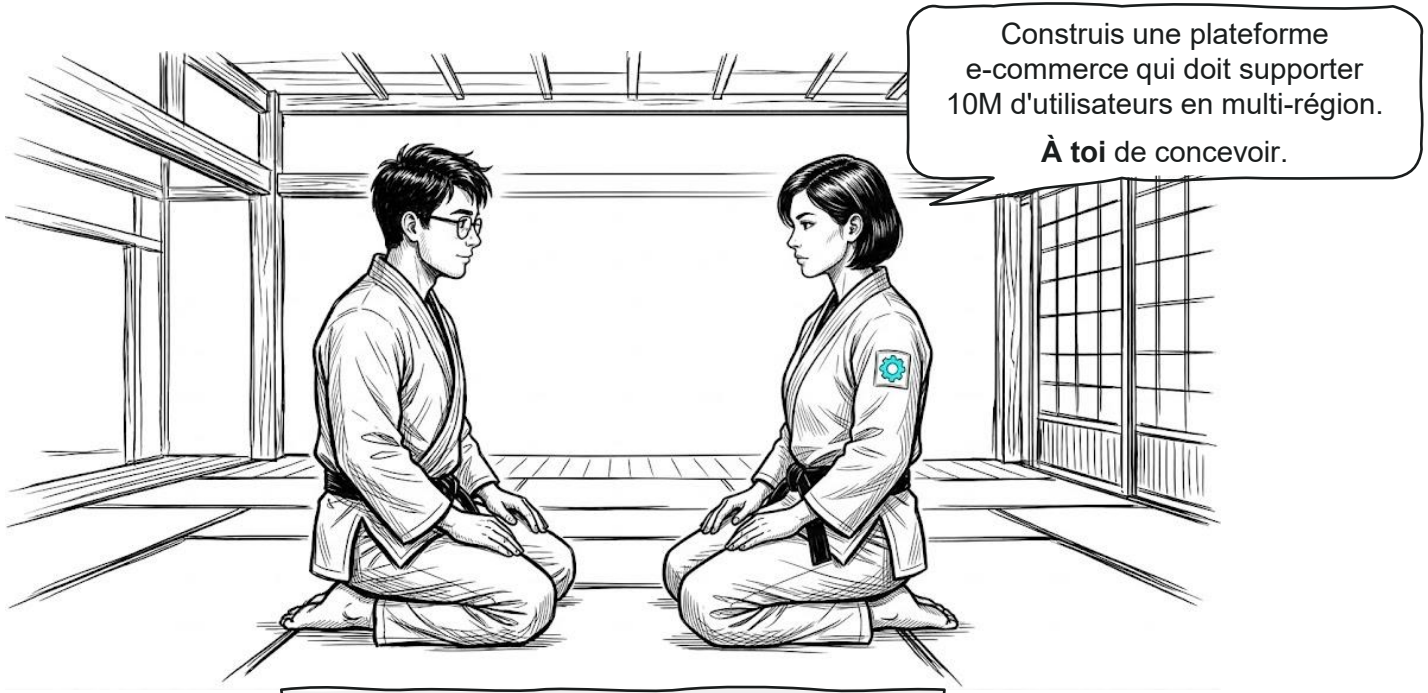
Le problème de l'apprentissage en 2040



Le junior a **validé** 100 choix d'architectures.
Mais n'en a vraiment **compris** aucun.

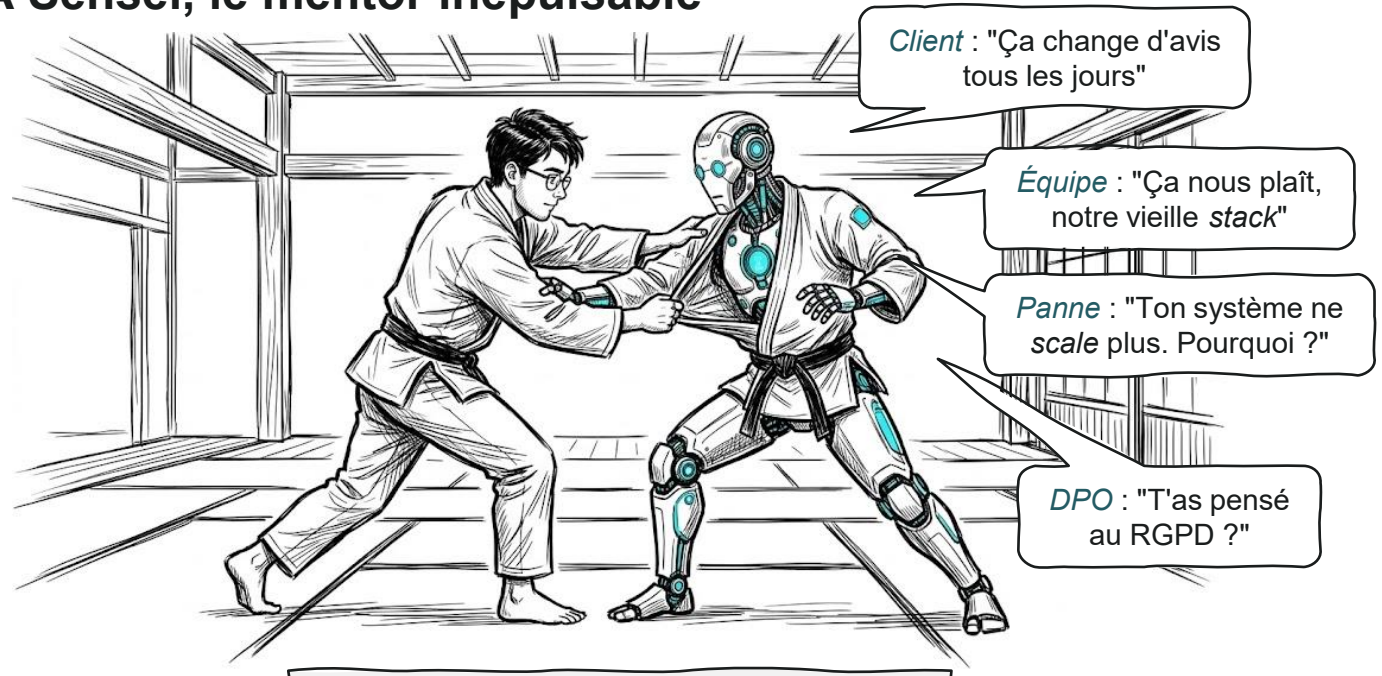
Les Katas d'Architecture

En 2020



Les Katas d'Architecture Augmentés

EN 2026: IA Sensei, le mentor inépuisable



Le junior doit **justifier** chaque décision.
C'est le vrai apprentissage.

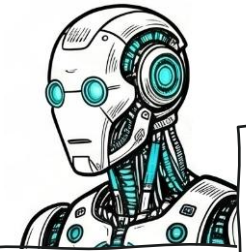
De « Vibe Coding » à « Vibe Architecting »

Compétences progressives



Vibe Coding

Je prompte l'IA, elle génère du code.
Je valide la syntaxe.



Vibe Spec

L'IA génère la spec et les tests.
Je challenge la spec avant.

Vibe Architecture

L'IA est mon *sparring partner*.
Je défends mes choix face à elle.
Je documente le « pourquoi ».





Maîtrise

Je sais quand faire confiance et
quand challenger l'IA.
Je forme les autres.

🎯 Conclusion

Retour vers le présent

Takeaways pour 2040

2040 Avant

L'IA a tué notre métier.
On n'a plus d'architectes.
Juste des orchestrateurs.
C'est un chaos.



2040 Après



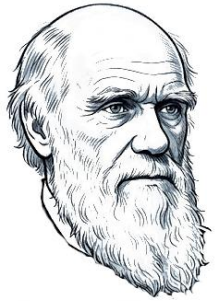
J'ai compris.
Vous avez les garde-fous.
Vous maintenez la culture du « pourquoi ».
Vous ne laissez jamais l'IA décider seule.

Qu'allez-vous perdre ou gagner ?

- ✅ Vitesse de documentation
- ✅ Extraction contextuelle (NFRs)
- ✅ *Sparring partner* 24/7
- ✅ Vulgarisation métier
- ✅ Formations augmentées
- ✅ Vérifications continues

- ❌ Réflexion superficielle
- ❌ Standardisation excessive
- ❌ Perte de « pourquoi »
- ❌ *Bus factor* sur la compréhension
- ❌ Dilution des responsabilités

Les gains l'emportent si et seulement si vous **gardez l'humain *in-the-loop***



Est-ce que Darwin a eu raison des architectes ?

Takeaways pour 2026



Toute résistance
est inutile



L'humain décide
(pas la machine)



Documentez le
« pourquoi » (toujours)

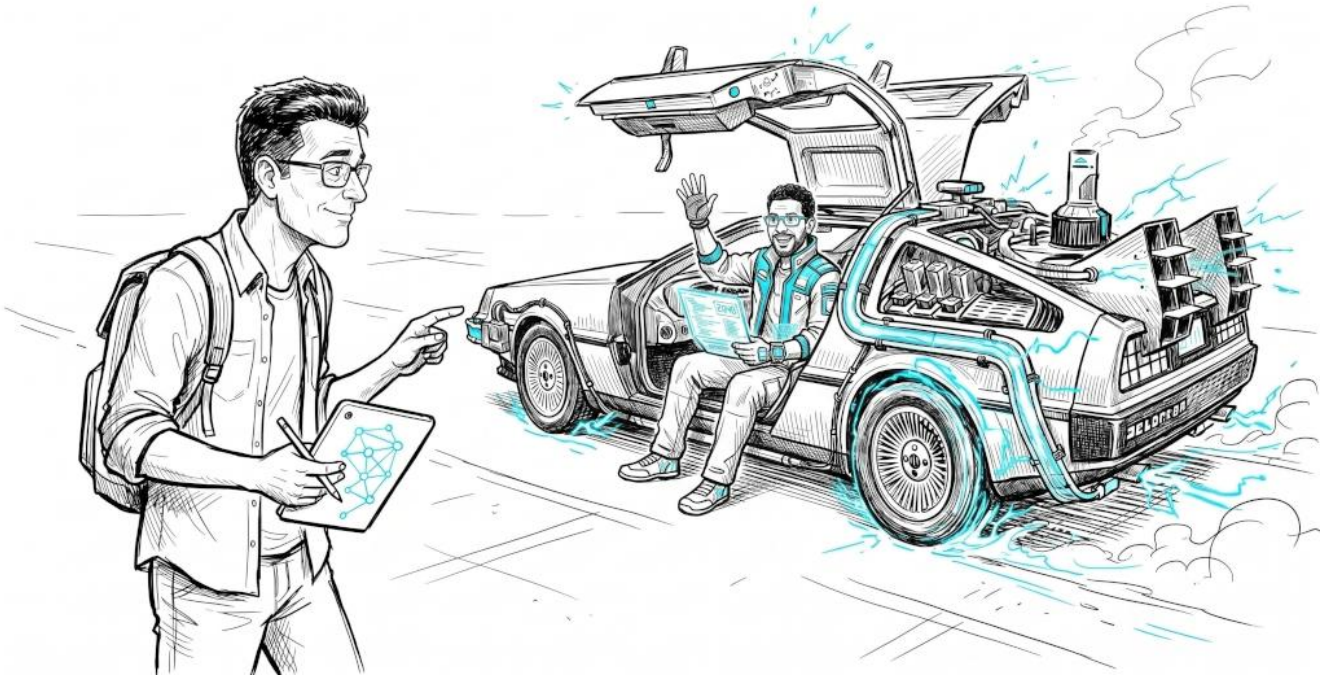


Maintenez toujours
le contexte



Testez les
suppositions de l'IA

That’s all folks!



That’s all folks!



That’s all folks!



Qui sommes-nous ?



Raphaël Semeteys

Tech Advocate

WORLDLINE

@raphiki.bsky.social

dev.to/raphiki

raphaelsemeteys



Alexandre Touret

Software Architect

MAGELLAN

@touret.info

blog.touret.info

alexandre-touret

2023

1 plateforme à concevoir
+ 2 architectes
= 3 possibilités

Raphaël SEMETEYS - Alexandre TOURET

2026

1 plateforme à concevoir
+ 2 architectes
+ IA
= 4 possibilités

Raphaël SEMETEYS - Alexandre TOURET

2040

1 plateforme à concevoir
+ 2 architectes
+ IA^{N}
= 100 possibilités ?

Raphaël SEMETEYS - Alexandre TOURET



Merci !